

Code-Layout, Readability and Reusability

Date	5th July 2026
Team ID	RAHMA HUSSEIN JUMA
Project Name	CREDIT CARD APPROVAL PREDICTION
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	All scripts use consistent 4-space indentation throughout
2	Proper File Structure	Files and folders are logically organize	Yes	Separate scripts per epic (EDA, preprocessing, model building, app), with templates/, static/, model/ folders in the Flask app
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. application_df, credit_df, merged_df, best_model, input_data
4	Function / Method Names	Functions are descriptively named	Partial	Key functions like is_risky() and Flask's home()/predict() are clear, but most preprocessing logic runs as plain top-level script code rather than named functions
5	Code Comments	Inline and block comments explain log	Yes	Each script has a header comment explaining the epic, plus inline comments before non-obvious steps (e.g. why EMPLOYED_YEARS placeholder values are masked)
6	Modular Design	Code is split into reusable functions/m	partial	The Flask app is modular (separate routes/functions); the data pipeline scripts are mostly linear/procedural rather than broken into reusable functions, which was a deliberate simplicity trade-off given the deadline
7	No Redundant Code	Duplicate or unused code is removed	Yes	No leftover unused variables or duplicate logic across scripts
8	Error Handling	Exceptions and errors are handled gra	No	No try/except blocks currently; invalid form input or missing files would raise an unhandled error rather than a friendly message

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technolog	yWhere Reused	Reusability Level (High / Medium / Low)
1	best_model.pkl / scaler.pkl / model_columns.pkl	Python (pickle)	Loaded once in app.py, reused for every single prediction request without retraining	High

2	is_risky() function	python	Used in epic3_preprocessing.py to label every applicant's credit history	Medium
3	style.css	CSS	Shared across both index.html and result.html	High
4	Preprocessing steps (encoding, scaling logic)	Python (pandas, scikit-learn)	Logic conceptually reused between epic3_preprocessing.py (training time) and app.py's /predict route (inference time) — though currently duplicated rather than imported from a shared module	Low

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear separation by epic, consistent formatting
Readability	4	Simple, plain variable names and comments make intent easy to follow
Reusability	3	Model artifacts are properly reusable; preprocessing logic is duplicated between training and inference rather than shared
Documentation / Comments	4	Each script is well-commented at the header and key logic points
Overall Score	3.75	Solid, readable code appropriate for the project's scope; main gaps are error handling and factoring preprocessing into a shared, importable module